

TRADING BOT — INTERNAL DOCUMENTATION

Autonomous AI Trading Bot

Project overview, three production workflows, intelligence sources, lab, strategies & algorithms

Multi-lane equity, 24/7 crypto, and an options-wheel premium-seller — gated by data quality, compliance, VaR, and an LLM lab pinned to claude-opus-4-7.

VERSION	v2.0 — 2026-04-29
GENERATED	2026-04-29
REPOSITORY	/Users/bharathkandala/Trading
TRADING VENUE	Alpaca paper (https://paper-api.alpaca.markets)
LLM MODEL	claude-opus-4-7 — pinned across all roles
WORKFLOWS	3 live + 1 library-only (W1 stocks · W2 crypto · W3 wheel · W4 news-trader)

Contents

1	Project overview	scope, modules, CLI surface
2	System architecture	end-to-end data & control flow (Figure D1)
3	Workflow W1 — Stocks momentum	multi-lane equity (Figure D2)
4	Workflow W2 — Crypto 24/7	spot crypto loop (Figure D3)
5	Workflow W3 — Options-Wheel	CSP → assigned → CC state machine (Figure D4)
6	Workflow W4 — News-driven LLM trader	library-only (Figure D5)
7	News & intelligence sources	10+ feeds × workflow consumers
8	Strategies & algorithms	indicators, entries, exits, sizing
9	The Lab	walk-forward, PBO, DSR, Optuna, architect/reviewer (Figure D6)
10	Risk gates	22-stage pre-trade stack + VaR
11	Decision lifecycle	audit → DecisionStore → reconcile → digest (Figure D7)
12	Scheduler & daily timeline	every cron job (Figure D8)

CHAPTER 1 Project overview

The Trading Bot at a glance

An autonomous, paper-trading bot that runs three independent strategies — multi-lane US equity, 24/7 spot crypto, and an options-wheel — under a single orchestrator, decision store, and risk envelope.

Every order passes a layered gate stack (data quality, compliance, intel, risk caps, VaR, daily/weekly P&L circuit-breakers) before reaching the broker.

A nightly research lab runs purged walk-forward backtests, gates promotions on PBO and Deflated Sharpe, and uses a Claude Opus 4.7 architect/reviewer pipeline to draft new strategy templates.

Operational tooling — diagnose, daily digest, midday snapshot, dashboard, alert drain — keeps the human in the loop without coupling decisions to the human's response time.

Headline numbers

Metric	Value
Python modules	132 .py files in src/trading_bot/ (incl. backtest, dashboard, news_trader, options, roles, validation)
CLI subcommands	33 Click subcommands registered under bot (status, scan, full-run, wheel-scan, lab, ...)
Trading workflows	3 live (W1 stocks, W2 crypto, W3 options-wheel) + 1 library-only (W4 news-driven LLM)
Intel sources	10+ free / paid feeds (Alpaca News, Polygon, FRED, GDELT, SEC EDGAR, Finnhub, ApeWisdom, CoinGecko, alt.me, Truth Social)
LLM model	claude-opus-4-7 — pinned in anthropic_client.py and news_trader/news_controller.py
Persistence	SQLite (state.db) — append-only DecisionStore + Alembic-managed schema; closed_trades.db, backtestBars.db, news_sentiment.db caches
Scheduler	APScheduler with ~25 cron / interval jobs across stocks, crypto, wheel, lab, monitoring, reporting
Hosting	macOS launchd plists in ops/launchd/ — daemon, lab, dashboard, supervisor

CLI surface — 33 subcommands grouped by purpose

Group	Subcommands
Live trading	bot full-run · bot intel-scan · bot crypto-scan · bot wheel-scan · bot wheel-manage · bot wheel-close · bot iv-capture · bot wheel-universe-build
Reporting	bot status · bot daily-report · bot daily-digest · bot midday-snapshot · bot eod-report · bot rich-report · bot wheel-status · bot alert-drain
Research	bot screen-universe · bot backtest · bot rank · bot lab · bot lab-backfill · bot evolve
Operations	bot dry-run · bot reconcile · bot verify-stops · bot vip-scan · bot news-warm · bot massive-refresh · bot portfolio-watch · bot schedule-audit
Long-running	bot dashboard · bot daemon · bot supervisor

Source-tree map

```

Trading/
├── README.md
├── pyproject.toml
├── src/trading_bot/
│   ├── cli.py, orchestrator.py, daemon.py, supervisor.py
│   ├── strategy.py, strategy_lanes.py, screener.py, regime.py
│   ├── intelligence.py, intel_gates.py, intelligence_finnhub.py,
│   │   intelligence_apewisdom.py, news_sentiment.py, vip_tweets.py
│   ├── massive_client.py (Polygon) # universe + bars + news + sentiment + short interest
│   ├── alpaca_client.py # equities + crypto + options brackets
│   ├── risk_manager.py, var_engine.py, compliance.py, data_quality.py
│   ├── decisions_store.py, state_db.py, pnl_state.py, cost_tracker.py
│   ├── walkforward.py, evolution.py, calibration.py, lab.py
│   ├── validation/ (pbo.py, dsr.py, ... promotion gates)
│   ├── backtest/ (simulator.py, metrics.py, massive_bar_loader.py)
│   ├── news_trader/ (event_extractor.py, news_controller.py)
│   ├── options/ (wheel_runner.py, wheel_lane.py, chain.py, wheel_state.py)
│   ├── roles/ (strategy_architect, code_reviewer, calibrator, sentiment_analyst, param_optimizer)
│   ├── dashboard/ (FastAPI app + templates)
│   ├── reports.py, email_*.py, session_summary.py, journal_alpha.py
│   ├── scheduler_jobs.py, cadence.py
│   └── ...
├── strategy/ # config, watchlists, restricted lists
├── docs/ # this PDF + architecture.pdf + plans/specs
├── tools/build_doc_pdf/ # this builder
├── tests/ # pytest suite (100+ files)
├── migrations/versions/ # Alembic
├── ops/launchd/ # macOS daemons (daemon, lab, dashboard, supervisor)
├── runs/YYYY-MM-DD/ # JSONL execution logs
└── data/ # SQLite DBs + heartbeat.json

```

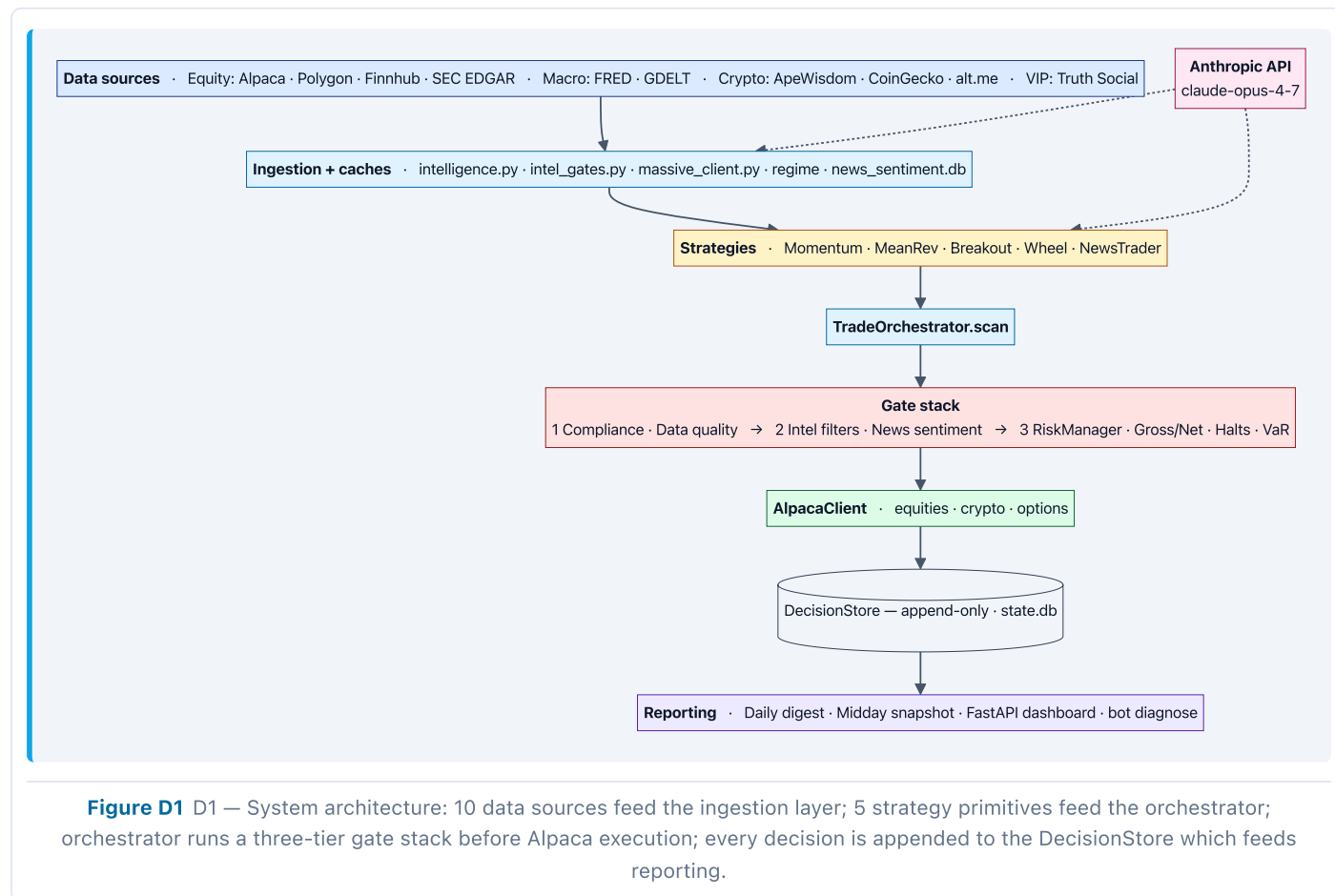
WHY THREE WORKFLOWS IN ONE PROCESS?

Each workflow has its own data cadence (1500-stock universe vs 24/7 crypto vs 50-symbol option chains), but they share the same orchestrator, DecisionStore, RiskManager, and Alpaca paper account. The shared envelope is what enforces gross/net caps and per-strategy halts across all three at once.

CHAPTER 2 System architecture

End-to-end data & control flow

The bot is structured as a small set of long-lived processes (the trading `daemon`, the `lab`, the `dashboard`, and a `supervisor`) sharing a SQLite state database, an append-only `DecisionStore`, and a single Alpaca paper account. Every workflow flows through the same orchestrator and risk envelope.



Process layout

Four launchd plists in `ops/launchd/` bring up the system:

- **daemon** — the live scheduler (`APScheduler`) that runs every cron job in §12.
- **lab** — independent scheduler for the research loop (§9).
- **dashboard** — FastAPI UI that reads `state.db` read-only.
- **supervisor** — restarts any of the above on watchdog failure (`watchdog_account.py`, `watchdog_stall.py`).

WHY ONE ORCHESTRATOR FOR THREE WORKFLOWS?

All three live workflows share `RiskManager` and `DecisionStore`, which means a per-strategy halt or a global gross/net cap applies across them in one place. `TradeOrchestrator.scan` reads the asset class from the candidate and picks the right gate path, but the envelope is shared.

CHAPTERS 3 – 6 The four workflows

Workflow comparison

Three of the four workflows are live in the scheduler today. W4 (news-driven LLM trader) is library-only and gated behind an explicit wiring step.

#	Workflow	CLI trigger	Cadence	Asset class	LLM in path?	Output
W 1	Stocks momentum / multi-lane equity	bot intel-scan, bot full-run, bot rich-report	* /N min, 09:00–15:00 ET, Mon–Fri	US equities	Lab only — entries are deterministic	Alpaca bracket orders + DecisionStore + email digest
W 2	Crypto 24/7	bot crypto-scan	Every 30 min, 24/7 (IntervalTrigger)	Spot crypto (USD-quoted)	Lab only	Alpaca crypto orders + DecisionStore
W 3	Options-Wheel (CSP→assigned→CC)	bot wheel-scan, bot wheel-manage, bot wheel-universe-build, bot iv-capture	Daily 10:15 ET; manage every 30 min 10:00–15:00 ET; build 21:30 ET; IV 09:45 ET	Equity options	No — fully deterministic state machine	sell_to_open / buy_to_close + WheelCycle state + alerts
W 4	News-driven LLM trader (library only)	Library callable propose_news_trade(); not in scheduler	Manual / future cron	Equities (event-driven)	Yes — claude-opus-4-7 in news_controller.py	Decision JSON → DecisionStore (when wired)

CHAPTER 3 Workflow W1 — Stocks momentum / multi-lane equity

Live

W1 · Stocks momentum / multi-lane equity

The flagship US-equity workflow. Pre-market builds the universe and ranks 1500+ tickers; through the day, scanners run multi-lane signal generation, intel gates, and risk checks before submitting bracket orders to Alpaca paper.

CLI TRIGGER	bot intel-scan · bot full-run · bot rich-report
SCHEDULE	stock_scanner (CronTrigger every N min, 09:00–15:00 ET) · premarket_rank (07:30 ET) · massive_refresh (06:30 ET) · midday_rerank (12:00 ET)
ASSET CLASS	US equities
LLM IN PATH?	Not in entry path — entries are deterministic indicators. LLM enters only at the lab/evolution layer (StrategyArchitect, CodeReviewer, Calibrator) on Saturdays via roles/strategy_architect.py.

End-to-end sequence

- 06:30 ET — massive_refresh refreshes Polygon grouped bars and the active universe.
- 07:30 ET — premarket_rank re-scores candidates and writes strategy/opportunities.md.
- Every N min, 09:00–15:00 ET — stock_scanner job calls bot intel-scan.
- _load_active_universe pulls top-25 stocks from the ranked watchlist.
- Stage-1 screener (screener.build_stage1_shortlist) computes 1d / 5d / volume score relative to SPY.
- Stage-2 lanes (Momentum / MeanReversion / Breakout) emit RankedCandidate with conviction.
- Live regime detection via FRED VIX (regime.detect_regime).
- TradeOrchestrator.scan iterates per symbol:
 - W2b restricted-list (compliance.check_restricted) → skip if restricted
 - Idempotency check (existing position / pending order / already-traded-today)
 - Bar fetch + MIN_BARS_FOR_INDICATORS
 - W2a data-quality (check_bar_freshness, check_completeness)
 - MomentumStrategy.evaluate emits Signal with stop_loss + take_profit
 - News-sentiment floor (news_sentiment.passes_filter)
 - _news_intel_gate runs stock_earnings_gate, stock_insider_cluster_gate, macro_shock_gate
 - RiskManager.check enforces per-trade-risk %, max-position %, concentration, asset-class regime cap, gross/net cap, daily/weekly halt, per-strategy halt
 - AlpacaClient.place_order_with_stop_loss submits a bracket order.
 - TradeJournal.append + RiskAfter populated.
 - DecisionStore.append persists the audit, compliance, data-quality, alerts, execution-constraints JSON columns.
- 16:05 + 21:55 ET — Reconciler reconciles fills against Alpaca account.
- 16:30 ET — daily_digest email summarises trades, P&L, and rejections.

Data sources consumed

- Polygon grouped daily bars (massive_client.py)

- Alpaca news (intelligence.py)
- FRED VIX (regime detection)
- GDELT macro shock
- Finnhub earnings + insider
- SEC EDGAR Form 4 (insider RSS)
- Truth Social VIP RSS (alert-only)

Strategies / signal sources

- MomentumStrategy
- MeanReversionStrategy (currently disabled in router)
- MomentumLane
- MeanReversionLane
- BreakoutLane

Outputs

- Alpaca bracket orders (entry + stop-loss)
- DecisionStore.append for every reject/place
- TradeJournal entries
- Daily digest email at 16:30 ET

Pre-market · 06:30 ET massive_refresh → 07:30 ET premarket_rank writes opportunities.md

Scan loop · cron */N min, 09:00–15:00 ET, Mon–Fri

Universe + screener · load_active_universe top-25 stocks → Stage-1 1d/5d/vol vs SPY

Stage-2 lanes · MomentumLane · MeanReversionLane · BreakoutLane (regime via FRED VIX)

Gate 1 fail-closed · restricted-list · idempotency · data-quality (freshness + completeness)

Gate 2 fail-open · news-sentiment floor · earnings · insider cluster · macro shock

Gate 3 fail-closed · RiskManager.check (per-trade · concentration · gross/net · halts · VaR)

Execution · AlpacaClient.place_order_with_stop_loss (bracket)

Persist · DecisionStore.append + TradeJournal.append

Reporting · 16:05 + 21:55 Reconciler → 16:30 daily_digest email

Figure D2 D2 — Workflow W1 stocks momentum: pre-market universe build → scan loop with Stage-1/2 ranking → 3-tier gate stack → bracket order → DecisionStore → reconcile + digest.

CHAPTER 4 Workflow W2 — Crypto 24/7

Live

W2 · Crypto 24/7

A continuous crypto loop that mirrors the equity orchestrator path but with crypto-specific universe discovery, data-quality thresholds, and crypto-only intel gates.

CLI TRIGGER	bot crypto-scan
SCHEDULE	crypto_scanner (IntervalTrigger every 30 min — no day-of-week filter)
ASSET CLASS	Spot crypto (USD-quoted)
LLM IN PATH?	Not in entry path. Same lab/evolution layer as equities. There is also a crypto_scanner role file (roles/crypto_scanner.py).

End-to-end sequence

1. Every 30 min — crypto_scanner job calls bot crypto-scan (24/7).
2. `_load_active_universe(crypto_only=True)` pulls auto-discovered crypto symbols from `discover_crypto_universe`.
3. Universe filtered to USD-quoted, non-stablecoin, not in `strategy/crypto_blocklist.yaml`.
4. `TradeOrchestrator.scan` with crypto-specific data-quality thresholds (`max_bar_age_hours_crypto`).
5. Bypasses news-sentiment floor (Polygon news is equity-focused).
6. Crypto-specific intel gates run:
 7. `crypto_fear_greed_gate` — blocks outside [`floor=20`, `ceiling=80`]
 8. `crypto_reddit_spike_gate` — blocks if mentions $\geq 2\times$ 24h-prior
 9. `crypto_coingecko_gate` — blocks if `sentiment_votes_up_percentage < 50`
 0. `macro_shock_gate` — applies cross-asset
 1. `RiskManager.check` with crypto asset-class caps.
 2. `AlpacaClient` submits crypto market order.
 3. `DecisionStore.append`; alerts only on action.

Data sources consumed

- Alpaca crypto bars (`alpaca_client.py`)
- `alternative.me` Crypto Fear & Greed (`intel_gates.py`)
- `ApeWisdom r/CryptoCurrency` (`intelligence_apewisdom.py`)
- `CoinGecko sentiment_votes_up_percentage` (`intel_gates.py`)
- `GDELT` macro shock (cross-asset)

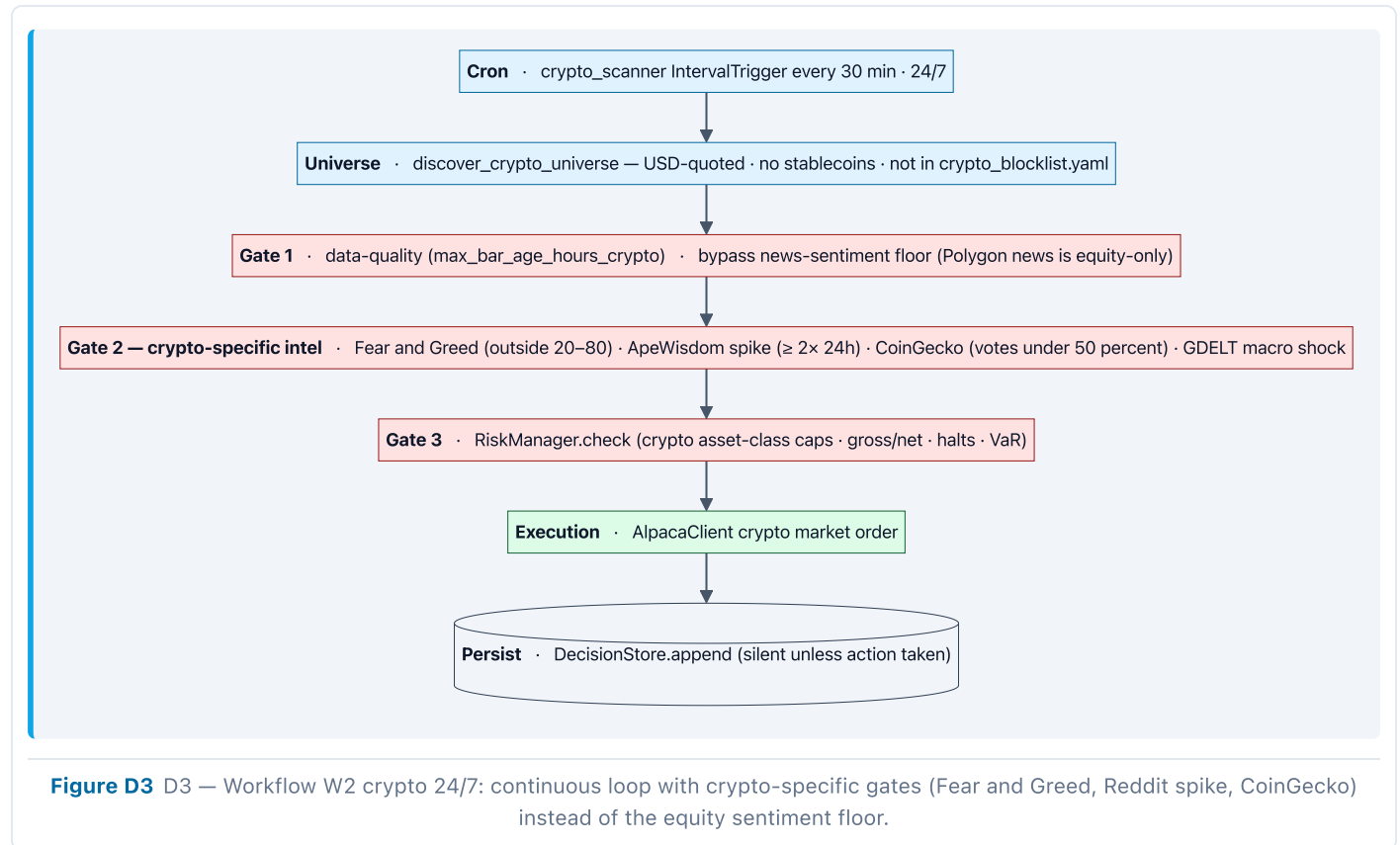
Strategies / signal sources

- `MomentumStrategy` under crypto regime mapping (`strategy_for_regime`)

Outputs

- Alpaca crypto market orders
- `DecisionStore.append`

- Silent unless action taken (no email per scan)



CHAPTER 5

Workflow W3 — Options-Wheel (CSP → assigned → CC)

Live

W3 · Options-Wheel (CSP → assigned → CC)

A premium-selling wheel that sells cash-secured puts on quality names, holds the stock when assigned, then sells covered calls at strike $\geq$ cost basis. State-machine driven, no LLM.

CLI TRIGGER	bot wheel-scan · bot wheel-manage · bot wheel-close · bot iv-capture · bot wheel-universe-build · bot wheel-status
SCHEDULE	wheel_universe_build (21:30 ET nightly) · iv_capture (09:45 ET weekdays) · wheel_scan (10:15 ET weekdays — single daily entry) · wheel_manage (every 30 min 10:00–15:00 ET weekdays)
ASSET CLASS	Equity options
LLM IN PATH?	None — fully deterministic. LLM only via shared lab/evolution layer.

End-to-end sequence

- 21:30 ET nightly — wheel_universe_builder walks Alpaca optionable equities, applies allowlist + Finnhub mcap/listing-age filters, writes wheel_universe_cache.
- 09:45 ET weekdays — iv_capture mass-fetches option chains, writes option_iv_history rows.
- 10:15 ET weekdays — wheel_scan runs once for the day.
- Per symbol gather: regime, VIX, sentiment_score, IV rank, ApeWisdom WSB spike, Finnhub earnings, current WheelCycle row.
- WheelLane.passes_preflight: wheel enabled; cycle phase $\in$ {None, assigned}; regime $\in$ {trending_up, sideways}; VIX in [floor, ceiling]; sentiment $\geq$ floor; IV rank $\geq$ 30; no WSB spike; no earnings within dte_max+2 days.
- Stage-3 contract pick: pick_csp_contract or pick_cc_contract by delta band (typ. 0.20–0.30) and DTE band.
- Risk gates: option_collateral_ok (allocation cap + per-symbol concentration); _sector_cap_check (default 25%).
- OptionAlpacaClient.sell_to_open at limit-bid; cycle state recorded via wheel_state.open_csp / open_cc.
- Every 30 min 10:00–15:00 ET — wheel_manage iterates active cycles:
 - Take-profit at $\text{mid} \leq (1 - \text{take_profit_pct}) \times \text{credit}$
 - Force-close at $\text{dte} \leq \text{dte_force_close}$
 - Roll if $|\text{delta}| \geq \text{delta_breach}$ AND $\text{rolls_used} < \text{max_rolls_per_cycle}$

Data sources consumed

- Alpaca option chains + IV (chain.py via OptionAlpacaClient)
- Finnhub earnings calendar + market cap + listing age
- ApeWisdom WSB spike (wheel preflight)
- Polygon news sentiment (preflight floor)
- option_iv_history table (IV rank)

Strategies / signal sources

- WheelLane: Phase.NONE → CSP_OPEN → ASSIGNED → CC_OPEN → CLOSED

Outputs

- sell_to_open / buy_to_close option orders
- OptionFill rows in state.db
- WheelCycle state rows
- AlertEvents (wheel_csp_opened, wheel_cc_opened, wheel_take_profit, wheel_dte_close, wheel_roll, wheel_allocation_cap)

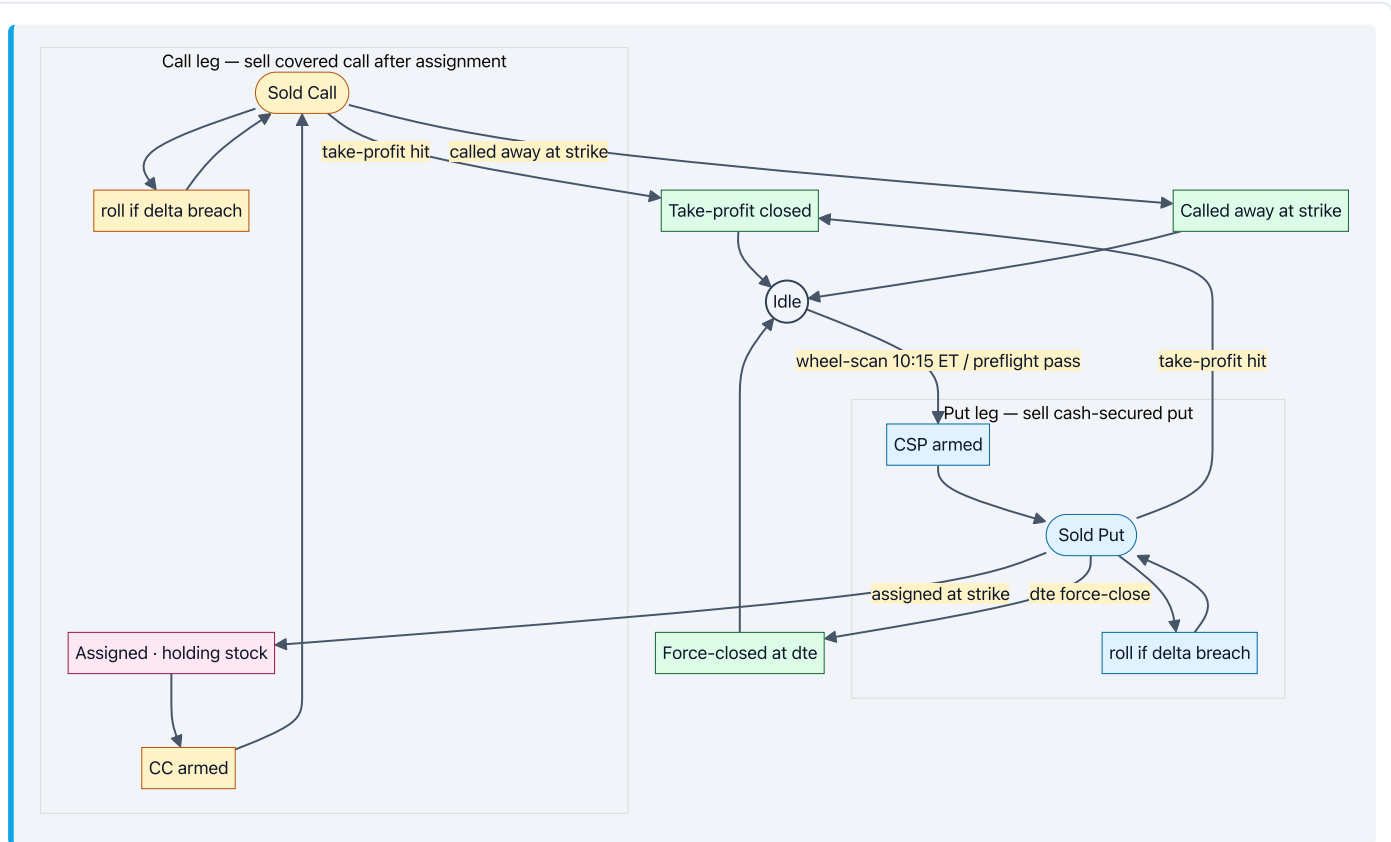


Figure D4 D4 — Workflow W3 wheel state machine: cash-secured put → assignment → covered call premium-selling, with rolls, take-profit, and force-close branches.

CHAPTER 6 Workflow W4 — News-driven LLM trader

Library only

W4 · News-driven LLM trader

A discretionary news-event trader powered by Claude Opus 4.7. Detects events with regex, packages context for the LLM, and parses a strict JSON action — currently library-only, not yet wired into the scheduler or CLI.

CLI TRIGGER	Library callable: propose_news_trade(NewsControllerInput, llm_call_fn, model='claude-opus-4-7')
SCHEDULE	None — not registered in scheduler_jobs.py
ASSET CLASS	Equities (event-driven)
LLM IN PATH?	Yes — pinned to claude-opus-4-7 at news_trader/news_controller.py:38.

End-to-end sequence

1. Caller assembles a NewsControllerInput (symbol, market state, position, recent news cluster, primary-filing flag, novelty score).
2. event_extractor.extract_event runs pure-regex classification — no LLM call yet.
3. news_controller.propose_news_trade builds the system prompt (NEWS_CONTROLLER_PROMPT_VERSION = 'v1').
4. AnthropicClient.complete is called with model='claude-opus-4-7'.
5. Strict JSON output is parsed; on parse error → fail-closed Decision(action='no_trade').
6. _ACTION_MAP translates LLM action to orchestrator action string (enter, scale_in, reduce, exit, hedge, no_trade, escalate_to_human).
7. Returned Decision flows through RiskManager.check (when wired) before any order is placed.

Data sources consumed

- Same intel feed as W1 (Alpaca News, Polygon news, SEC EDGAR Form 4 RSS, GDELT)
- Event clusters classified by lexicon (merger, earnings, guidance, litigation, leadership_change, dividend, product_launch)

Strategies / signal sources

- Allowed action set: NO_TRADE, ENTER, SCALE_IN, REDUCE, EXIT, HEDGE, ESCALATE
- Capital cap (default 5 %) and required time-stop

Outputs

- Decision dataclass (action, sizing, stop, time_stop, reason) → DecisionStore (when wired)

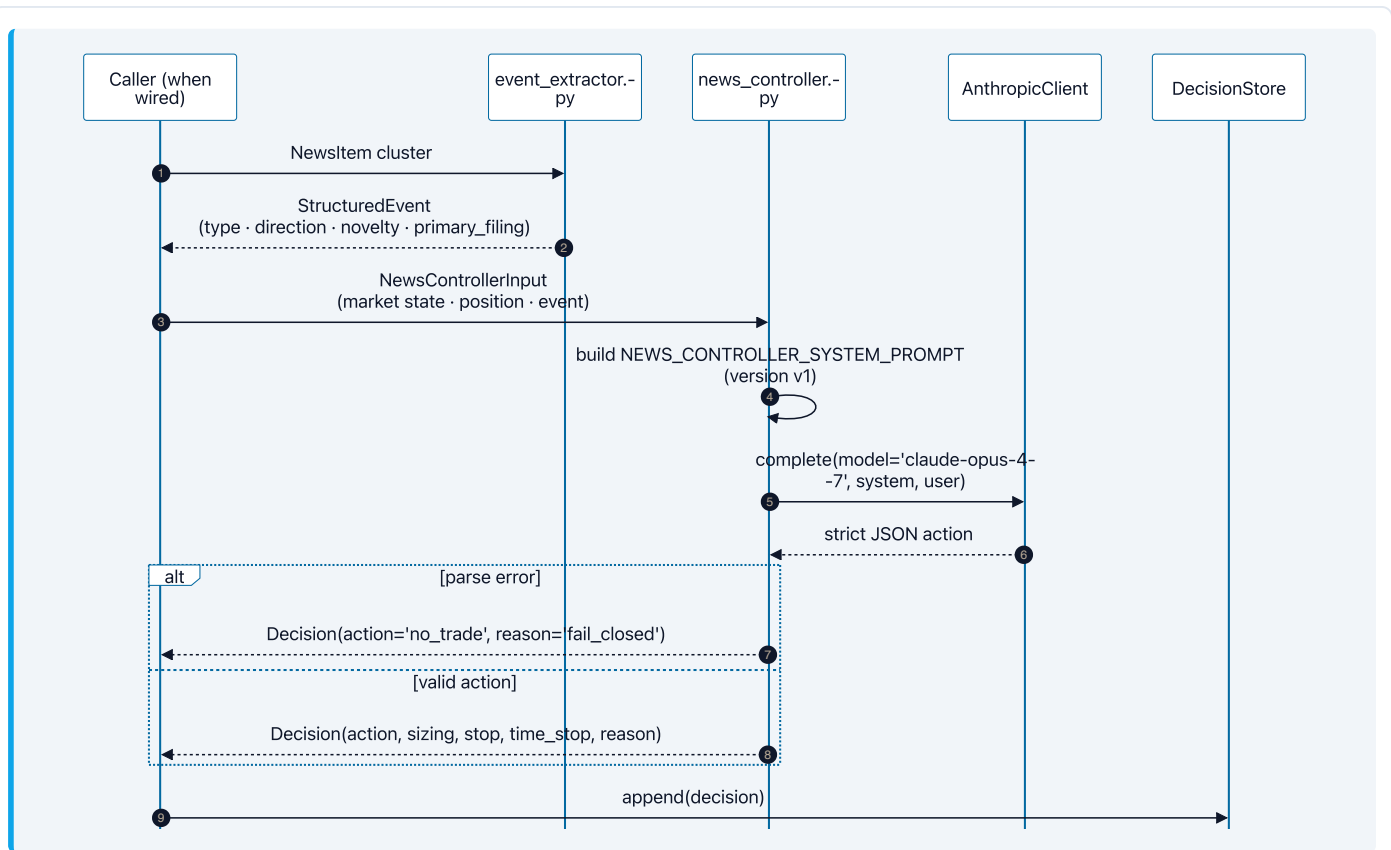


Figure D5 D5 — Workflow W4 news-driven LLM trader: regex extracts event, controller builds prompt, Claude Opus 4.7 returns strict JSON, fail-closed on parse error.

STATUS: LIBRARY-ONLY

`news_trader/news_controller.py:propose_news_trade` is callable but is not registered in `scheduler_jobs.py` and has no Click subcommand. Wiring requires (a) a cron entry that builds `NewsControllerInput` from the live news cluster and (b) a path through `TradeOrchestrator` to enforce risk gates on the returned `Decision`.

CHAPTER 7 News & intelligence sources

Every feed the bot consumes

The bot's intelligence layer is split across a baseline aggregator (intelligence.py), a per-trade gate framework (intel_gates.py), a Polygon REST client (massive_client.py), and dedicated single-source modules. Compliance and data-quality checks fail closed; intel filters fail open by design — when a free feed is degraded, the trader continues with a logged warning rather than seizing up.

fail-open — If the feed errors or returns no data, the gate passes — the orchestrator does not block trading on a degraded free feed.

fail-closed — If the feed errors, the gate blocks — used for data-quality and compliance only.

Source	Module / class	Endpoint / API	Cadence / TTL	Consumed by	Fail mode
Alpaca News	intelligence.AlpacaNews.for_symbols	alpaca-py NewsClient (per-symbol financial news)	On-scan, 2-day lookback, 3–5 items / symbol	W1 stocks (digest + news_trader cluster)	fail-open
Polygon grouped daily bars	massive_client.MassiveClient.daily_grouped	GET /v2/aggs/grouped/locale/us/market/stocks/{date}	Pre-market 06:30 ET; throttled MIN_CALL_INTERVAL_S=13s; backoff (10,30,60,120,300)	W1 universe + Stage-1 screener	fail-closed
Polygon news + sentiment	massive_client.MassiveClient.news / aggregate_sentiment	GET /v2/reference/news?ticker=... (with insights)	On-scan, 50-item batches, 3-day lookback	W1 (news-sentiment floor) + W4 cluster	fail-open
Polygon short interest	massive_client.MassiveClient.short_interest	GET /stocks/v1/short-interest (FINRA biweekly)	Biweekly	W1 (squeeze flag, optional)	fail-open
FRED macro	intelligence.get_macro_snapshot	fred.stlouisfed.org/graph/fredgraph.csv (VIXCLS, DGS10, DFF) — anonymous	Pre-scan + 09:25 ET refresh	W1 + W3 (regime detection, MacroSnapshot)	fail-open
GDELT 2.0 doc API	intelligence.get_gdelt_events	api.gdeltproject.org/api/v2/doc — query 'stock market OR S&P 500 OR Federal Reserve', mode ArtList, timespan 24H, max 8	30 min TTL	W1 + W2 (macro_shock_gate)	fail-open
SEC EDGAR Form 4	intelligence.get_recent_insider_filings	sec.gov/cgi-bin/browse-edgar?action=getcurrent&type=4 (atom)	Hourly during scan window	W1 surface — visible in digest	fail-open

Source	Module / class	Endpoint / API	Cadence / TTL	Consumed by	Fail mode
Finnhub earnings calendar	<code>intelligence_finnhub.FinnhubClient + intel_gates.stock_earnings_gate</code>	GET /api/v1/calendar/earnings (lookahead_days=5)	24 h TTL	W1 + W3 wheel preflight	<code>fail-open</code>
Finnhub insider transactions	<code>intelligence_finnhub.FinnhubClient + intel_gates.stock_insider_cluster_gate</code>	GET /api/v1/stock/insider-transactions (90-day window)	24 h TTL	W1 (block at ≥ 5 insider sells)	<code>fail-open</code>
alternative.me Crypto Fear & Greed	<code>intel_gates.crypto_fear_greed_gate</code>	GET api.alternative.me/fng	1 h TTL	W2 only — block outside [20, 80]	<code>fail-open</code>
ApeWisdom (Reddit)	<code>intelligence_apewisdom.ApeWisdomClient + intel_gates.crypto_reddit_spike_gate</code>	GET apewisdom.io/api/v1.0/filter/CryptoCurrency, /wallstreetbets, /stocks	5 min TTL	W2 (crypto spike block) + W3 (WSB spike preflight)	<code>fail-open</code>
CoinGecko	<code>intel_gates.crypto_coingecko_gate</code>	GET api.coingecko.com/api/v3/coins/{id} (sentiment_votes_up_percentage)	30 min TTL	W2 only — block if < 50 % up-votes	<code>fail-open</code>
Truth Social VIP	<code>vip_tweets.VipScanResult</code>	Truth Social RSS for handles in strategy/vip_handles.yaml	On-scan	W1 alert-only (HIGH/MED/LOW keyword tiers)	<code>fail-open</code>
Anthropic — Claude Opus 4.7	<code>anthropic_client.AnthropicClient</code>	anthropic-py Messages API (model='claude-opus-4-7')	Lab calls (Saturday architect/reviewer); news_controller (when wired)	Lab + W4	<code>fail-closed (cost-)</code>

Per-source endpoint, refresh cadence, consuming workflow, and fail mode.

COMPLIANCE VS. INTEL
intel_gates.py is fail-open: a degraded free feed never blocks trading. compliance.py and data_quality.py are fail-closed: a missing restricted-list file or a stale bar always blocks.

COST HALT
AnthropicClient cannot call the model when cost_tracker.is_halted(session) returns true. It raises BudgetExceededError instead — protecting the user from a runaway lab loop.

CHAPTER 8 Strategies & algorithms

From RSI to a 7-action LLM controller

Five strategy primitives drive the three live workflows. Equity entries are deterministic indicator-based; the wheel is a state-machine premium-seller; the news trader is the only LLM-mediated path.

Strategy	File	Type	Indicators / Signal	Entry rule	Exit rule	Sizing
MomentumStrategy	src/trading_bot/strategy.py:24	Trend-follow, long	RSI(14), MACD line vs signal, EMA(20), 5-day return	RSI $\in$ [55, 70] AND MACD > signal AND close > EMA20 AND 5d > 0	Stop = max(EMA20, entry $\times$ (1 – 5%)); TP = 2 $\times$ R; max-hold 60 d (backtest)	Fixed-fractional Kelly: min(equity $\cdot$ 0.5%/per-share-risk, equity $\cdot$ 4.5%/entry); ROUND_DOWN to integer shares
MeanReversionStrategy	src/trading_bot/strategy.py:108	Mean-reversion, long	RSI(14), EMA(20), 5-day return	RSI $\in$ [25, 35] AND close $\leq$ EMA20 $\cdot$ 1.01 AND 5d $\geq$ –5%	Stop = entry $\times$ (1 – 4%); same TP / max-hold	Same as Momentum (currently disabled in router — PF 0.66 in sideways regime)
MomentumLane (Stage-2)	src/trading_bot/strategy_lanes.py:41	Multi-lane scorer	RSI, MACD spread, relative-5d	RSI $\in$ [55,70], MACD > signal, close > EMA20, 5d > 0	Sets entry/stop/TP via shared sizing	Conviction 0.4–0.9 from RSI position + MACD spread + relative-5d
MeanReversionLane (Stage-2)	src/trading_bot/strategy_lanes.py:89	Multi-lane scorer	RSI, Bollinger Bands(20, 2 σ)	RSI < 30 AND last < lower Bollinger band	Same shared sizing path	Conviction grows with (lower – last)/lower $\times$ 5, capped 0.85
BreakoutLane (Stage-2)	src/trading_bot/strategy_lanes.py:127	Breakout	20-day high, volume vs 20-day average	close > prior 20-day high AND last_volume $\geq$ 1.5 $\times$ 20d avg	Same shared sizing path	Conviction = 0.5 + min((vol_ratio – 1.5) $\times$ 0.2, 0.4)
WheelLane (Options)	src/trading_bot/options/wheel_lane.py + wheel_runner.py	Premium-selling state machine	Regime, VIX, sentiment, IV rank, earnings calendar, WSB spike	Phase $\in$ {None, assigned} AND all preflight gates pass; pick contract by delta + DTE band	Take-profit at (1 – take_profit_pct) $\times$ credit; force-close at dte $\leq$ dte_force_close; roll if delta $\geq$ breach	Capped by allocation.option s_max_pct + per-symbol concentration cap + sector cap (default 25 %)
NewsTrader (W4)	src/trading_bot/news_trader/news_controller.py	LLM event-trader	Event class (regex), direction, novelty, primary-filing flag	LLM picks ENTER / SCALE_IN / REDUCE / EXIT / HEDGE / ESCALATE; else NO_TRADE	LLM-issued time-stop; required, fail-closed if missing	Capital cap default 5 % of equity (capital_cap_pct)

Indicators, entry/exit rules, and sizing per strategy primitive.

Regime router

strategy_for_regime (strategy.py:168) decides which strategy fires given the live regime. MeanReversion is currently disabled across all regimes — PF 0.66 in sideways was losing.

Live regime	Strategies that fire
trending_up	MomentumStrategy + MomentumLane + BreakoutLane
sideways	MomentumLane + BreakoutLane (MeanReversion disabled)
trending_down	Long-only halt — no equity entries
risk_off	Block all entries (gate stack rejects)

STAGE-1 SCORE
screener.py:60 → $1d_return \times 1.4 + relative_5d + \min(vol_ratio, 3) \times 2$ — benchmark-relative against SPY 5-day move.

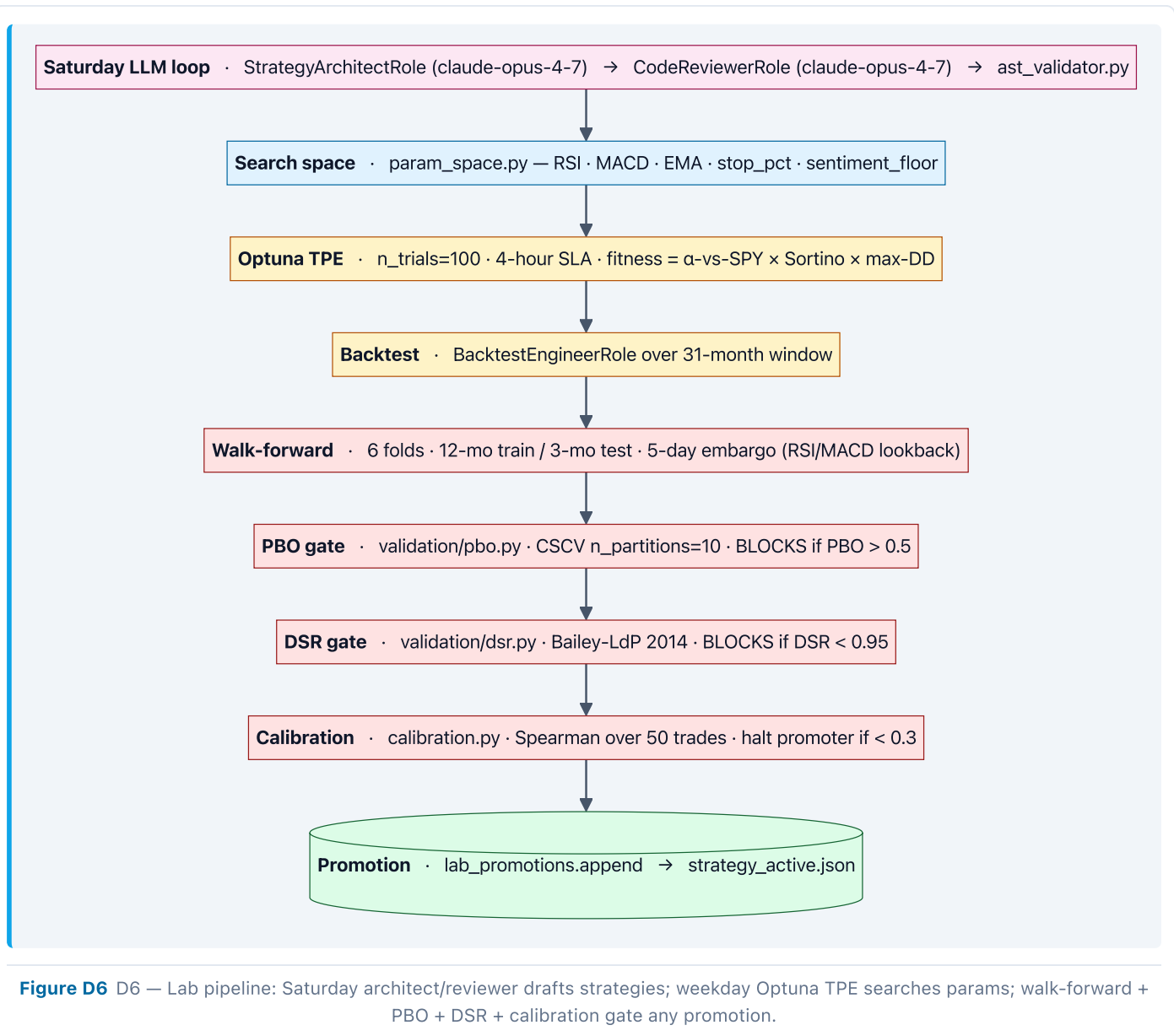
WHEEL STATE MACHINE
Idle → CSP-armed → Sold-Put → Assigned → CC-armed → Sold-Call → Called-Away → Idle. Rolls are self-loops on Sold-Put or Sold-Call when delta breaches and rolls_used < cap.

CHAPTER 9

The Lab — research & auto-promotion

Walk-forward, PBO, Deflated Sharpe, and an LLM architect

The Lab (src/trading_bot/lab.py) is a long-running launchd daemon that reproduces and stress-tests strategies on historical data. Its goal is to push only statistically meaningful changes into the live config — and to refuse promotions that look good in-sample but fail honest out-of-sample tests.



Components

Purged walk-forward

`src/trading_bot/walkforward.py` · entry: `default_folds, walk_forward_backtest`

- Default 6 folds — 12-month train + 3-month test, walking by `test_months=3`.
- Embargo gap defaults to 5 calendar days in `walk_forward_backtest` (`walkforward.py:197`) — enough to break leakage from RSI(14)/MACD(26)/EMA(20) lookbacks.
- `FoldDefinition` supports configurable embargo per call; production sets 5d explicitly.

Probability of Backtest Overfitting (PBO)

`src/trading_bot/validation/pbo.py` · entry: `probability_of_backtest_overfitting`

- Implements Bailey & López de Prado 2014 CSCV (Combinatorially Symmetric Cross-Validation).
- $n_partitions = 10 \rightarrow C(10, 5) = 252$ IS/OOS combinations.

- For each combination, picks the IS-Sharpe argmax and tests whether its OOS rank $> N/2$.
- Returns the fraction of combinations where the IS-best is below median OOS — promotion gate: $PBO > 0.5$ BLOCKS.

Deflated Sharpe Ratio (DSR)

`src/trading_bot/validation/dsr.py` · entry: `deflated_sharpe_ratio`

- Implements Bailey & López de Prado 2014 — penalises Sharpe by the number of trials and the higher moments of returns.
- $\text{sharpe_max_estimate} = (1 - \gamma) \cdot \Phi^{-1}(1 - 1/N) + \gamma \cdot \Phi^{-1}(1 - 1/(N \cdot e))$ with $\gamma \approx 0.5772$ (Euler-Mascheroni).
- $z = (\text{SR}_{\text{obs}} - \text{SR}_{\text{max}}) \cdot \sqrt{(n-1)} / \sqrt{(1 - \text{skew} \cdot \text{SR} + (\text{kurt}-1)/4 \cdot \text{SR}^2)}$ → return $\Phi(z)$.
- Pure-Python normal CDF / inv-CDF — no scipy dependency. Promotion gate: $\text{DSR} > 0.95$.

Optuna TPE evolution

`src/trading_bot/evolution.py` + `roles/param_optimizer.py` · entry: `optimize_strategy`

- `optuna.create_study(direction='maximize', sampler=TPESampler())` — `n_trials=100`, 4-hour SLA budget per run.
- Search space (`param_space.py`): `rsi_lower` (50–60), `rsi_upper` (65–75), `ema_period` (15–30 int), `stop_pct` (3.0–7.0), `sentiment_floor` (–1.0–0.0).
- Each trial calls `BacktestEngineerRole` over a 31-month window with `n_folds=6`.
- Fitness = `compute_fitness(alpha_vs_spy_x, sortino, max_dd_pct)` — combines all three into one scalar.

Calibration drift

`src/trading_bot/calibration.py` · entry: `compute_drift_score`

- Spearman rank correlation (Pearson on ranks) between predicted edge and realised P&L.
- Severity bands: > 0.5 ok · $0.3-0.5$ warning · < 0.3 high → halt Promoter for 7 days.
- Below 10 samples → 'insufficient_data', no halt.

Architect / Reviewer pipeline

`src/trading_bot/roles/strategy_architect.py` + `roles/code_reviewer.py` · entry: `Saturday cron 06:00 ET`

- `StrategyArchitectRole` asks `claude-opus-4-7` for 1–3 strategy templates as strict JSON (name, rationale, expected_regime, code, tests, params_to_search).
- Hard constraints: no `os/sys/subprocess/requests/eval/exec/open/__import__`; must implement `evaluate(symbol, ind, equity) → Signal` and `from_params` classmethod.
- AST-validated by `ast_validator.py`; `CodeReviewer` LLM second pass; only then does Optuna search over `params_to_search`.

Promotion gates (in order)

Auto-promote (cron 02:45 ET) takes a candidate from `lab_promotions` and applies these gates in order. Any failure rejects.

Gate	Threshold
Walk-forward folds passed	≥ 4 of 6 with positive Sharpe and max-DD better than incumbent
PBO	≤ 0.5 — strictly less than 50% probability of being a lucky overfit
Deflated Sharpe	≥ 0.95 — significant after deflation by trial count and higher moments
Calibration drift	Spearman ≥ 0.3 over a 50-trade window when prior data exists
Risk regression	$\text{max-DD} \leq \text{incumbent_max_dd} \times 1.10$ (no worse than 10% looser drawdown)
AST validator	code under proposal cannot import <code>os/sys/subprocess/requests/eval/exec/open/__import__</code>
Code review	claude-opus-4-7 reviewer must return APPROVED in strict JSON

Lab scheduler jobs

Job	Cron (ET)	What it does
param_search	02:00 daily	Optuna TPE search over each enabled strategy's params space
auto_promote	02:45 daily	Apply the promotion gates above; on success update strategy_active.json
calibrate	05:00 daily	Compute Spearman drift on the trailing 50 trades; halt promoter if < 0.3
saturday_evolve	06:00 Sat	Architect → Reviewer → Optuna → walk-forward → promotion candidate

WHY TWO LLM PASSES?

The Architect proposes new strategy code; the Reviewer is a separate Opus 4.7 call that grades the proposal against a checklist (no banned imports, evaluate signature, params_to_search ranges). Two independent calls reduce the risk of a single-shot hallucination passing the AST validator.

CHAPTER 10 Risk gates — the 22-stage pre-trade stack

What every order has to clear before it reaches the broker

Every prospective order traverses a layered gate stack before it can reach Alpaca. Compliance and data-quality gates fail closed; intel and risk gates degrade gracefully. The whole stack is summarised below in execution order.

fail-closed — gate blocks on error (compliance, data quality, risk math).

fail-open — gate passes on error (free intel feeds; logged warning).

#	Gate	Fail mode	File	What it checks
1	Restricted-list (W2b)	fail-closed	compliance.py:check_restricted	Reject if symbol is on strategy/restricted_list.yaml.
2	Approved venue	fail-closed	compliance.py:check_approved_venue	Only Alpaca paper URL is accepted.
3	Idempotency	fail-closed	orchestrator.py — existing position / pending order / already-traded-today	Block re-entry the same trading day; block when there is an open order or open position.
4	Data-quality (W2a)	fail-closed	data_quality.py:check_bar_freshness, check_completeness	Bar age within thresholds (RTH 48 h, overnight 96 h, crypto 2–6 h, intraday 0.25–1 h); required OHLC columns present; NaN ratio $\leq 5\%$.
5	News-sentiment floor	fail-open	news_sentiment.py:passes_filter	Equities only — daily aggregate sentiment must clear the floor; crypto bypasses (Polygon news is equity-focused).
6	Stock earnings	fail-open	intel_gates.py:stock_earnings_gate	Block if earnings within lookahead_days = 5 (Finnhub).
7	Stock insider cluster	fail-open	intel_gates.py:stock_insider_cluster_gate	Block at ≥ 5 insider sells in trailing 90 days (Finnhub).
8	Macro shock	fail-open	intel_gates.py:macro_shock_gate	Aggregated GDELT sentiment ≤ -3.0 blocks all entries.
9	Crypto F&G	fail-open	intel_gates.py:crypto_fear_greed_gate	Crypto only — block outside [floor 20, ceiling 80].
10	Crypto Reddit spike	fail-open	intel_gates.py:crypto_reddit_spike_gate	Crypto only — block if mentions $\geq 2 \times 24$ -prior.
11	Crypto CoinGecko	fail-open	intel_gates.py:crypto_coin_gecko_gate	Crypto only — block if sentiment_votes_up_percentage < 50 .
12	Per-trade risk %	fail-closed	risk_manager.py:_check_per_trade_risk	Notional $\times$ stop-distance $\leq$ per_trade_risk_pct $\times$ equity.
13	Max position %	fail-closed	risk_manager.py:_check_max_position	Notional $\leq$ max_position_pct $\times$ equity.
14	Concentration	fail-closed	risk_manager.py:_check_concentration	Per-symbol notional $\leq$ max_symbol_concentration_pct.
15	Asset-class regime cap	fail-closed	risk_manager.py:_check_asset_class_caps	Stocks/crypto/options caps conditioned on the live regime.

#	Gate	Fail mode	File	What it checks
16	Gross / net exposure	fail-close	risk_manager.py:_check_gross_net	Gross ≤ 200 % of equity; net ≤ 100 % of equity.
17	Daily / weekly P&L	fail-close	risk_manager.py:_check_daily_weekly_limits	Trip when daily_pnl_pct ≤ -daily_loss_limit_pct or weekly equivalent.
18	Per-strategy halt	fail-close	risk_manager.py – RiskState.halted_strategies	Skip orders whose strategy_name is in the halted set; other lanes continue.
19	VaR after-trade	fail-close	var_engine.py:var_after_trade	Historical VaR α=0.05, additive (no correlation credit) — must remain within configured cap.
20	Options collateral	fail-close	risk_manager.py:option_collateral_ok	Wheel-only — caps aggregate options collateral and per-symbol concentration.
21	Sector cap	fail-close	wheel_runner.py:_sector_cap_check	Wheel-only — equity + pending option collateral ≤ sector_cap_pct (default 25 %).
22	LLM cost halt	fail-close	cost_tracker.py:is_halted	AnthropicClient refuses to call the model when monthly spend ≥ 100 %.

Execution order is top-to-bottom. Each row is a guarded check inside TradeOrchestrator.scan or the wheel runner.

VaR engine — historical, additive, deliberately conservative

VaR is the conservative tail used in pre-trade and post-trade risk reporting. The implementation is deliberately additive — no correlation credit — so the limit always overestimates risk rather than understates it.

Function	File	Behaviour
historical_var(returns, alpha=0.05)	var_engine.py:17	Magnitude of the bottom-α empirical quantile loss; 95 % default.
expected_shortfall(returns, alpha=0.025)	var_engine.py:41	Mean of the worst α tail (CVaR). Reported alongside VaR.
var_after_trade(positions, candidate)	var_engine.py:62	Adds the candidate's marginal VaR to the portfolio total — no correlation netting.

LAYERED HALTS

Three independent kill switches act at different scopes: **per-strategy halt** (other lanes keep trading), **daily/weekly P&L circuit breaker** (whole portfolio), and **LLM cost halt** (refuses any Anthropic call when monthly spend hits 100%). The last one protects the lab from runaway loops.

CHAPTER 11 Decision lifecycle

From signal to digest — a single decision's path

The DecisionStore (`decisions_store.py` + `state_db.Decisions`) is append-only and the canonical record of every accept and reject. `bot diagnose SYMBOL` reads from it; the daily digest summarises it; reconciliation reconciles fills against it.

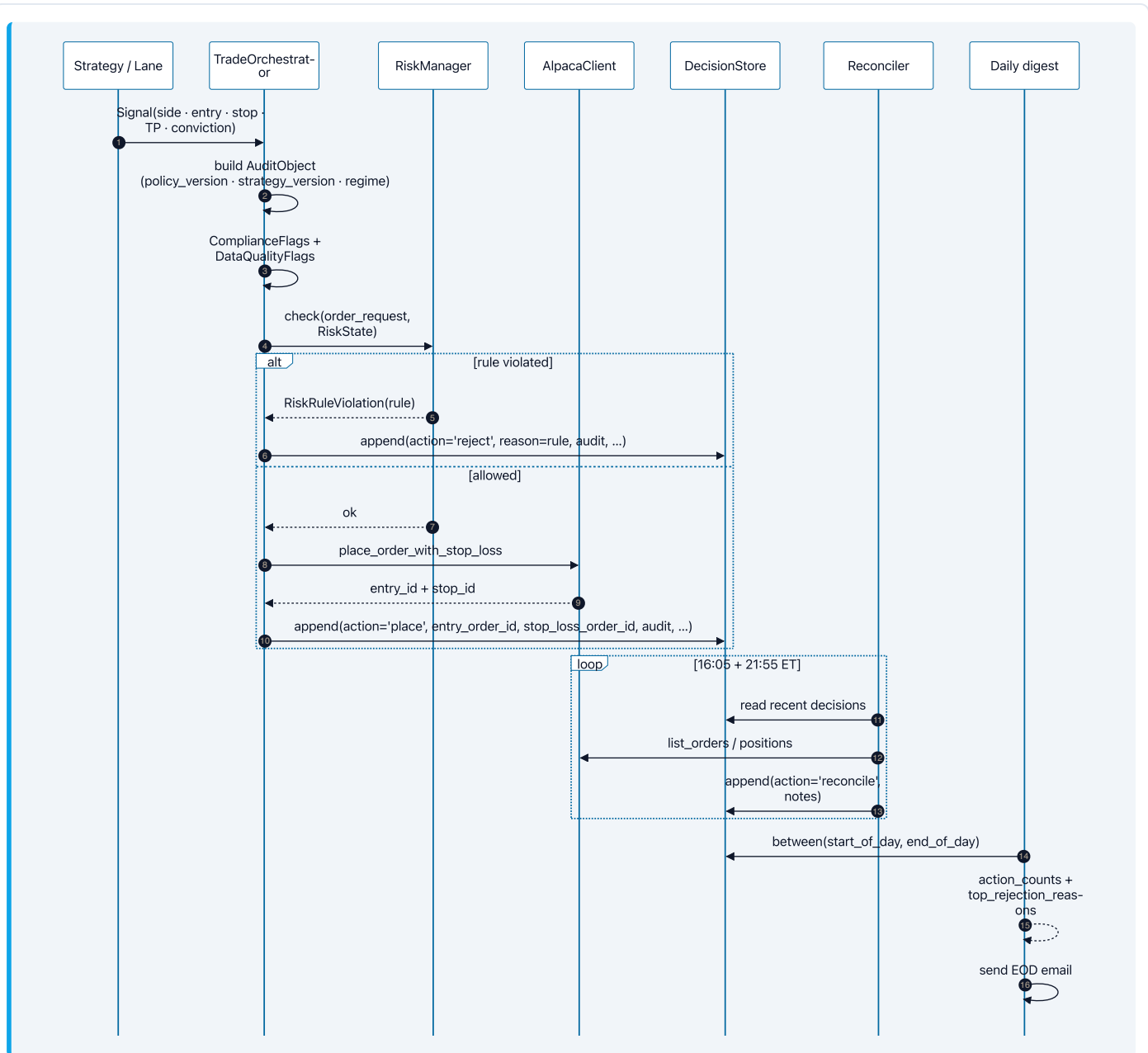


Figure D7 D7 — Decision lifecycle: every reject and every place is appended to the DecisionStore with full audit + compliance + data-quality JSON; reconciler and digest both read from the same store.

DecisionStore schema (state_db.Decisions)

Column	Type	What it stores
decision_id	string	d_<UTC>_<hex>
timestamp_utc	datetime	Decision time, UTC
symbol	string	Ticker / crypto pair / option underlying
action	string	place · reject · reconcile · scale_in · exit
reason	string	Human-readable reason (rule name on reject)
strategy	string	momentum · mean_reversion · breakout · wheel · news_trader

Column	Type	What it stores
regime	string	Live regime at decision time
asset_class	string	stocks · crypto · options
confidence	float	Lane conviction (0.0 – 1.0)
expected_edge_bps	float	Strategy-estimated edge in basis points
risk_after_json	json	VaR, gross, net, per-strategy halts after this trade
compliance_json	json	Restricted-list hits, approved-venue check
data_quality_json	json	Bar freshness, completeness, snapshot_id
execution_constraints_json	json	Limits, time-stop, max slippage
alerts_json	json	AlertEvents emitted for this decision
audit_json	json	policy_version, strategy_version, regime, full inputs
entry_order_id	string	Alpaca order ID (entry) on place
stop_loss_order_id	string	Alpaca order ID (stop) on place

APPEND-ONLY BY DESIGN

`decisions_store.py` exposes only `append`, `recent`, `between`, `action_counts`, and `top_rejection_reasons`. There is no update or delete API — once a decision is recorded, it stays. This is what makes the diagnose tool, the digest, and the lab calibration loop trustworthy retrospectives.

CHAPTER 12 Scheduler & daily timeline

Every cron job that keeps the bot alive

`scheduler_jobs.register_jobs` (`scheduler_jobs.py`) wires APScheduler over America/New_York and registers ~25 jobs across stocks, crypto, options, monitoring, and reporting. The Lab daemon registers its own scheduler (`param_search` 02:00 ET, `auto_promote` 02:45 ET, `calibrate` 05:00 ET, `saturday_evolve` Sat 06:00 ET).

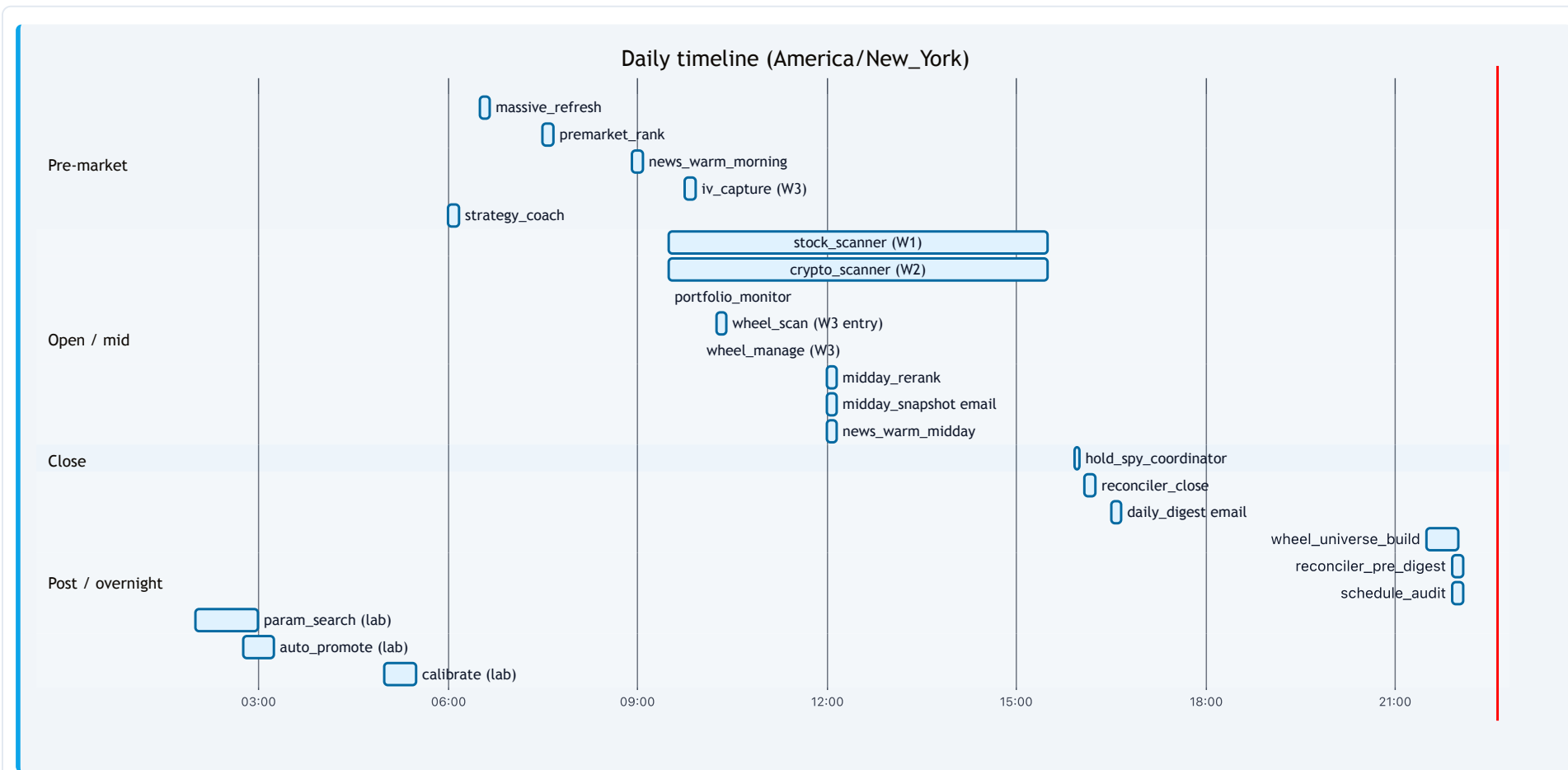


Figure D8 D8 — Daily timeline of scheduled jobs in America/New_York. Pre-market builds the universe and warms caches; open/mid runs scanners and the wheel manage loop; post wraps the day with reconciliation, digest, and the overnight lab cluster.

Full job list

Job	Trigger	Time / cadence (ET)	Purpose
heartbeat	interval	60 s, 24/7	Watchdog ping; writes data/heartbeat.json
stock_scanner	cron	*/*N 09:00–15:00 Mon–Fri	bot intel-scan — W1 entries
crypto_scanner	interval	30 min, 24/7	bot crypto-scan — W2 entries
portfolio_monitor	cron	09:00–16:00 Mon–Fri	Position monitor; alert detection
order_steward_sweep	cron	:20, :50 24/7	verify_stops — repair missing/stale stops
vip_listener	cron	09:00–16:00 Mon–Fri	Truth Social VIP RSS scan
news_warm_morning	cron	08:55 Mon–Fri	Warm Polygon news cache before open
news_warm_midday	cron	12:00 Mon–Fri	Mid-session news refresh
massive_refresh	cron	06:30 Mon–Fri	Polygon grouped daily bars + universe
premarket_rank	cron	07:30 Mon–Fri	Stage-1 score; writes strategy/opportunities.md
midday_rerank	cron	12:00 Mon–Fri	Re-score using running daily bars
midday_snapshot	cron	12:00 Mon–Fri	Email midday digest
daily_digest	cron	16:30 Mon–Fri	Email EOD digest
log_rotation	cron	Sun 03:00	Compress and rotate runs/ logs
strategy_coach	cron	06:00 Mon–Fri	30-day alpha-vs-SPY check; flip to fallback flag if losing
hold_spy_coordinator	cron	15:55 Mon–Fri	5-day exit/reverse on coach flag flip
schedule_audit	cron	21:55 24/7	Self-test that every job is registered and recently fired
alert_drain	interval	1 min, 24/7	Email throttling; flush AlertEvents queue
reconciler_close	cron	16:05 Mon–Fri	Reconcile newly closed trades vs Alpaca
reconciler_pre_digest	cron	21:55 24/7	Final reconciliation before EOD report
wheel_universe_build	cron	21:30 24/7	Nightly Alpaca optionable + Finnhub filter
iv_capture	cron	09:45 Mon–Fri	Mass-fetch chains; write option_iv_history
wheel_scan	cron	10:15 Mon–Fri	Single daily wheel entry pass
wheel_manage	cron	*/30 10:00–15:00 Mon–Fri	Take-profit / dte / roll for active cycles
param_search (lab)	cron	02:00 daily	Optuna TPE per strategy
auto_promote (lab)	cron	02:45 daily	Apply walk-forward / PBO / DSR / risk gates
calibrate (lab)	cron	05:00 daily	Spearman drift score over trailing 50 trades

Job	Trigger	Time / cadence (ET)	Purpose
saturday_evolve (lab)	cron	Sat 06:00	Architect → Reviewer → Optuna → walk-forward

All jobs registered by scheduler_jobs.register_jobs (live daemon) and the lab's own scheduler.

Time-of-day buckets

Bucket	Range	Jobs
Pre-market	06:30 → 09:25 ET	<code>massive_refresh</code> · <code>premarket_rank</code> · <code>news_warm_morning</code> · <code>iv_capture</code> · <code>strategy_coach</code>
Open	09:30 → 10:00 ET	<code>stock_scanner</code> · <code>crypto_scanner</code> · <code>portfolio_monitor</code>
Mid-day	10:00 → 15:30 ET	<code>wheel_scan (10:15)</code> · <code>wheel_manage (* / 30)</code> · <code>midday_rerank (12:00)</code> · <code>midday_snapshot (12:00)</code> · <code>news_warm_midday</code>
Close	15:30 → 16:00 ET	<code>hold_spy_coordinator (15:55)</code> · <code>stock_scanner final</code> · <code>crypto_scanner</code>
Post / overnight	16:00 → 06:30 ET	<code>reconciler_close (16:05)</code> · <code>daily_digest (16:30)</code> · <code>wheel_universe_build (21:30)</code> · <code>reconciler_pre_digest (21:55)</code> · <code>schedule_audit (21:55)</code> · <code>param_search (02:00)</code> · <code>auto_promote (02:45)</code> · <code>calibrate (05:00)</code> · <code>saturday_evolve (Sat 06:00)</code>

WHAT IF A JOB IS MISSED?

`schedule_audit` runs at 21:55 ET 24/7 and emits an alert if any expected job hasn't fired in its window. `order_steward_sweep` runs at `:20` and `:50` every hour and repairs any bracket missing its stop.